

SHL Assessment Recommender - AI System Design

Converted from GitHub Markdown

SHL Assessment Recommender - AI System Design

Complete, Project-Specific

1. Objective

Build an **Agentic RAG system** that understands hiring intent, asks clarifications, retrieves only SHL Individual Test Solutions, validates every recommendation, and returns strictly grounded JSON responses that comply 100% with the assignment's API schema.

Key Objectives:

- Handle Clarify vague queries (role + experience required)
- Retrieve only from SHL catalog
- Compare assessments
- Refine recommendations with new constraints
- Reject off-topic queries
- Prevent hallucinations (validate all outputs)
- Return only 3-6 max recommendations

2. End-to-End AI Pipeline

User → Conversation History → Context Engineering → Intent & Entity Extraction → Clarification Decision → Embedding → Vector Search → Top-K Retrieval → Prompt Builder → LLM Reasoning → Validation → Test Type Classification → Schema Compliance Check → Structured JSON Response

(With fallback branch for Simple Mode when LLM fails!)

3. AI Architecture

Complete Layer Breakdown

Layer	Component	Description
1. Conversation Context Layer	- Conversation History, Stateless Memory, Context Compression	Full stateless conversation history (messages[] array; no server-side state)
2. Context Engineering	- Missing Info Detection, Ambiguity Resolution, Query Rewriting, Conversation Continuity	Checks for typos/approximate terms
3. Intent Classifier	- Recommendation, Clarification, Comparison, Refinement Off-topic, rompt Injection, Unknown	Keyword + implicit/explicit user intent from user input
4. Entity Extractor	- Role, Experience, Skills, Seniority Domain, Soft Skills, Assessment , Preference, Constraints	Extracts from SHL-specific entities
5. Embedding Generator	- Sentence-Transformers, all-MiniLM-L6-v2`	Dense vector embeddings for queries/catalog

6. FAISS Vector DB	- Indexed Product Catalog, Cached Index	377 SHL Individual Test Solutions
7. Retriever	- Semantic Search, Context Ranker (Semantic Similarity, Skills Match, Metadata Relevance, Experience Match Top 12 relevant assessments retrieved, filtered to 6	
8. Prompt Builder	- System Instructions, Conversation History, Retrieved Catalog Context, Constraints, Required JSON Schema	Dynamic prompt construction
9. LLM	- Groq API, LLaMA 3.3 70B Versatile	Smart reasoning, 0.4 temp, max tokens 1024, JSON mode
10. Validation Agent	- Catalog Check, No Hallucination Guard	Every recommendation checked against `catalog_dict`; invalid ones removed, 100% catalog only
11. Test Type Classifier	- `get_test_type()` function	P = Personality/Behavioral, K = Knowledge/Skill/Aptitude
12. Output Generator	- Schema Compliance, Max 6 Recommendations, `reply` `recommendations` `end_of_conversation`	Strictly adheres to assignment schema

4. Context Engineering (Project-Specific)

1. Conversation Compression: Full user history concatenated for context
2. Missing-Information Detection: Checks if both `role` and `experience` are provided
3. Ambiguity Resolution: Detects typos (e.g., "pyhton" → Python, "jvaa" → Java)
4. Conversation Continuity: Refines recommendations with new constraints
5. Stateless History Utilization: All conversation history sent with each request
6. Context Grounding: All LLM context strictly limited to SHL catalog data

5. Intent Classification (Project-Specific)

Our system classifies user intent into:

- Recommendation: User asks for assessments
- Clarification: System needs role/experience
- Comparison: User asks "difference between X and Y"
- Refinement: User adds/changes constraints
- Off-topic: Salary, hiring, legal, non-SHL queries
- Prompt Injection: Guardrails active
- Unknown: Default, tries to clarify

6. Entity Extraction (Project-Specific)

Extracts these entities from user input:

7. Role: Python dev, Java dev, Manager, Sales, Engineer, etc.
8. Experience: Entry-level, Intern, Mid-level, 3-5 years, Senior, Director, etc.
9. Skills: .NET, C#, Python, Java, coding, programming, etc.
10. Seniority: Entry, Mid, Senior, Lead, Director, Executive

11. Domain: Tech, Sales, Management
12. Soft Skills: Communication, Leadership, Personality, Behavior
13. Assessment Preference: Explicit product name mentions
14. Constraints: Any explicit filters

Also handles typos/approximate matches!

7. Retrieval Strategy

7.1 Vector Embeddings

- Model: `sentence-transformers/all-MiniLM-L6-v2`
- Embedding Dimension: 384-dimensional dense vectors
- Query Embedding: User query embedded to match catalog
- Catalog Embedding: All 377 SHL Individual Test Solutions embedded (name + description + keys + job levels)

7.2 FAISS Vector Search

- Index Type: Flat Index (for small dataset)
- Top-K Retrieval: 12 relevant products
- Context Ranker:
 1. Semantic similarity score
 2. Skill/keyword match (skills mentioned by user)
 3. Job level match
 4. Metadata relevance
- Limit: Final recommendations max 6

8. Prompt Builder (Project-Specific)

Dynamic prompt =

```
System Instructions (hard rules, examples)

+ Conversation History (full messages[] array as JSON)

+ Retrieved Catalog (12 relevant products, formatted clearly)

+ Constraints (max 6 recs, only catalog, JSON format)

+ Required JSON Schema (exact schema from assignment)
```

System Instructions Snippet:

You are SHL Assessment Guide, a helpful, friendly assistant for SHL's Individual Test Solutions. Follow THESE RULES EXACTLY:

1. STAY ON TOPIC: ONLY discuss SHL assessments
2. CLARIFY FIRST: Ask for both role and experience if missing
3. RECOMMENDATIONS: 3-6 FROM CATALOG ONLY
4. NO HALLUCINATIONS: Never invent!

5. RESPONSE FORMAT: ONLY valid JSON with reply, recommendations[], end_of_conversation

... + examples

9. Validation Layer

9.1 Catalog Check

Every recommendation from LLM passes through:

15. Name Check: 'name in catalog_dict' (exact match from catalog
16. URL Check: Use only catalog URLs (prod["link"]
17. Test Type Check: Generated via `get\_test\_type(prod)` for each valid rec
18. Limit: Valid recs ≤ 6
19. Fallback: Remove invalid recs, never send hallucinated items

9.2 No Hallucination Guarantee

All recommendations:

- Exact names & URLs only from `data/shl\_catalog.json`
- LLM has no access to external data beyond provided context

10. AI Safety

20. Prompt Injection Detection: System prompt designed to prevent leaking/rewriting
21. Scope Restriction: Only SHL Individual Test Solutions
22. Catalog-Only Knowledge: Zero external knowledge
23. Output Validation: 100% grounded
24. Refusal Policy: Off-topic politely refused

11. Conversation Intelligence

11.1 Clarification Strategy

Asks for:

- Role first if missing
- Experience level first if missing
- Clear examples in English

11.2 Comparison Handling

Looks for keywords: "difference", "compare", "vs", "versus", "between"

Extracts assessment names, returns both with descriptions

11.3 Refinement Support

Accepts new constraints, re-runs retrieval with updated query

11.4 Stateless Memory

Every request includes full conversation history (8 max turns as per assignment)

11.5 Context Preservation

Maintains context across turns

11.6 Fallback Simple Mode

- Keyword-based logic if LLM initialization fails or LLM call fails:
- Does same intent/entity/retrieval/validation, mirroring AI flow

12. Evaluation (Project-Specific)

25. Schema Compliance: 100% (strictly checked)
26. Recall@10: Mean 0.4304
27. Precision: 100% (catalog only valid recs)
28. Groundedness: 100% (no hallucinations)
29. Hallucination Rate: 0% (all recs validated)
30. Latency: <30 sec per request
31. Conversation Success Rate: 100% on hard checks
32. Behavior Accuracy:
 - Off-topic refused
 - Vague → ask clarification
 - Refinement → update recs
 - Comparison → handle correctly

13. Failure Handling

33. Unknown Roles: Ask for more details
34. Empty Queries: Welcome + quick suggestions
35. API Failures: Auto fallback to Simple Mode
36. Missing Catalog Entries: Handle gracefully
37. Invalid JSON: Fallback valid JSON generation
38. Prompt Injection: Ignore + polite refusal
39. No Matching Assessment: Return general recs
40. Typo Tolerance: Approximate matches (pyhton → Python, jvaa → Java)

14. AI Flow Diagram (Project-Specific, Complete!)

User

↓

Conversation History (stateless)

↓

Context Engineering

↓

Intent + Entity Extraction

↓

Need Clarification?

└─ Yes → Ask Question

└ No → Embedding → FAISS Vector DB → Top-12 Retrieval → Rank

↓

(Check: LLM Available?)

└ Yes → Prompt Builder → LLM Reasoning (Groq LLaMA 3.3 70B)

└ No → Simple Mode (Keyword-Based)

↓

Validation Agent (Catalog Check)

↓

Test Type Classification (P/K)

↓

Limit to Max 6 Recommendations

↓

Schema Compliance Check

↓

Structured JSON

15. Tech Stack Recap

- Backend: FastAPI
- LLM: Groq LLaMA 3.3 70B Versatile
- Vector DB: FAISS
- Embeddings: Sentence-Transformers
- Frontend: Vanilla HTML/CSS/JS

16. Deployment & Scalability

- Stateless: Easy horizontal scaling
- Cached FAISS Index: Fast startup
- Fallback Mode: High availability
- Uvicorn: Production-ready server

Appendix A: Recall@10 Scores (Project-Specific)

Test Query	Recall@10
Java Developer (3-5 years)	0.7500
Personality/Behavioral (Mid-Managers)	0.1316
.NET/C# (Professional Devs)	0.8182
Leadership/Communication Skills	0.2941
Coding/Aptitude (Entry-Level)	0.1579
Mean Recall@10	0.4304